

Assignment 2: Incremental SfM

Haeun Jeon (2023100086)

Dept. Computer Science and Engineering, Korea University

Introduction to Computer Vision (DATA302)

Prof. Gyeongsik Moon

Oct. 30, 2025

1. Implementation

<Pipeline Overview>

The SfM implementation in this assignment consists of two phases (initialization and iteration). Each phase proceeds through modular functions and a state-management structure. In the initialization phase, two seed images are used to estimate the first camera poses (extrinsic parameters) and build the initial 3D points. In the iterative phase, a new image is selected and registered (via PnP), additional triangulation is performed using new observations, and then a global bundle adjustment refines the entire model.

1.1 Data Pre-processing and Feature Matching

- **Frame extraction:** Sample frames from a video at a fixed FPS and store them in `runs/<run>/frames`.
- **Feature extraction & matching:** Detect keypoints and descriptors for each frame using a SIFT-based `FeatureBackbone`. For adjacent frame pairs, perform feature matching and refine matches using Lowe ratio test and cross-check filtering.
- **Match packaging:** From `.npz`, recover the original keypoint indices (`qidx / tidx`), remove duplicates, then store as a tuple (`view1_id, view2_id, pairs, pts1, pts2, kps*_all`) in a list. Sort the list in descending order based on a quality score approximated by the number of matches or their average distance.
- **Intrinsic / extrinsic parameter model**
 - **Intrinsic matrix K :** Initially estimated heuristically from image resolution (e.g., $f \approx 1.2 \times \max(W, H)$) and principal point (cx, cy) at the image center). The intrinsic parameters are later refined together during bundle adjustment (BA).

1.2 Global State (SfMPipeline)

- **cameras:** `List[(R, t)]`
Stores the extrinsic parameters of each camera (rotation and translation), indexed by internal camera ID. Updated during initialization and image registration.
- **view_id_map / inv_view_id_map:**
Bidirectional mapping between external view IDs (frame indices) and internal IDs (array indices).
- **points_3d:** `(N, 3)`
All 3D points triangulated so far.
- **point_tracks:** `(view_int, kp_idx) -> point_id`
Track table that links each 2D keypoint observation to its corresponding 3D point.
- **observations:** `(view_int, point_id, x, y)`
Accumulates all 2D observations for bundle adjustment.

1.3 [Initialization] Two-view Seeding: Matching → Decomposition → Triangulation

- 1) **Estimate F / E** (`sfm_4.SfMPipeline.initialize_from_two_view`, calls `two_view_reconstruction`)
 - Estimate Fundamental matrix F using RANSAC → compute Essential matrix $E = K^T F K$, enforce rank-2 (set last singular value to 0 via SVD).
 - Use `cv.recoverPose(E, pts1, pts2, K)` to recover R, t and obtain an inlier mask.
 - Perform geometric verification: number of inliers, parallax (median), ensure $\det(R) = +1$, normalize translation t , reject invalid cases.

2) Initialize seed cameras (`sfm_4.SfMPipeline.initialize_from_two_view`)

- Get internal camera indices `i0`, `i1`.
- Place the reference camera at $\begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix}$ and the second camera at $\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}$.
- Store seed inlier keypoint indices for later triangulation.

3) Seed triangulation (`sfm_4.SfMPipeline.triangulate_seed_points`)

- Convert points to normalized coordinates, triangulation code implemented in `triangulation_custom.py` using $\begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix}$ and $\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}$.
- Filter points by cheirality (positive depth) and reprojection error.
- Add valid 3D points to `points_3d`, insert observations, and update the track table.

1.4 [Iteration] Image Selection → PnP Registration → Additional Triangulation → Global BA

1) Selecting the next image

(`sfm_4.SfMPipeline.build_index`, `sfm_4.SfMPipeline.select_next_best_image`)

- From the matches with already registered cameras, count how many 2D–3D correspondences exist (i.e., keypoints already associated with 3D points).
- Selection criteria:
 - maximize the number of 2D–3D overlaps, and
 - ensure sufficient spatial distribution (spread) in the image.
- For the selected external view ID, build a `pending_pnp` package (lists of `pts3d`, `pts2d`, and IDs).

2) Estimating extrinsics via PnP (robustly)

(`sfm_4.SfMPipeline.build_pnp_correspondences`, `sfm_4.SfMPipeline.register_image_via_pnp`)

- Use **UPnP/EPnP** to obtain an initial pose estimate → prune correspondences using reprojection error and positive depth (robust scoring with Median + MAD).
- If enough inliers remain, refine using ITERATIVE PnP, then finalize the inlier set.
- Register the new camera ($\mathbf{R}$, $\mathbf{t}$) with an internal ID, add all inlier observations to observations, and update the track table.

3) Triangulating new points

(`sfm_4.SfMPipeline.triangulate_new_matches_from_view`)

- Collect 2D–2D matches between the new view and already registered neighboring views that do not yet have a corresponding 3D point.
- Form projection matrices for each pair and run `cv.triangulatePoints`, then filter via cheirality and reprojection error.
- Add valid points to `points_3d`, and record observations from both views.
- If a match overlaps an existing 3D track (only one side was 3D), attach the new observation after reprojection validation.

4) Global Bundle Adjustment

(`global_ba.global_bundle_adjustment`)

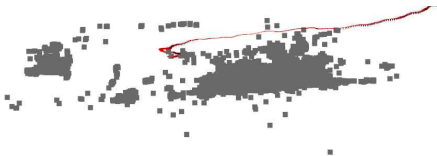
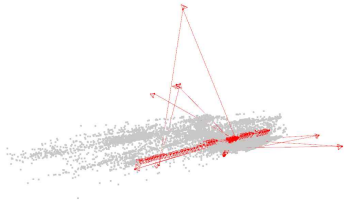
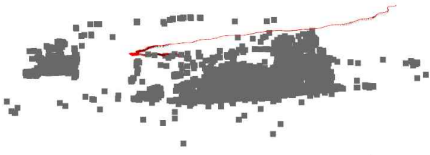
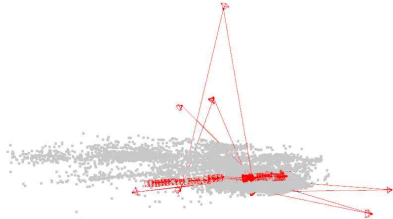
- Objective: minimize the reprojection error of all observations (robustified via Huber loss).
- Variables: all camera parameters ($\mathbf{R}$, $\mathbf{t}$) and intrinsic matrix $\mathbf{K}$ (initialized with $\mathbf{K}_0$); some elements may be fixed (anchor / `fix_point` options).
- Solver: Levenberg–Marquardt. After each BA iteration, update the intrinsic matrix in the pipeline (`sfm_4.SfMPipeline.set_intrinsics`).

((How to Run))

My SfM : `python main_4.py`

COLMAP : python colmap_comparison.py

2. Qualitative Results

reconstruction 하고자 했던 물체 (쇼파) - 동영상 중 한 프레임의 이미지만 가지고 옴	
	
COLMAP (서로 다른 위치에서 봄) 빨간색 선 : camera trajectory 회색 점 : 3D points	MY (서로 다른 위치에서 봄) 빨간색 선 : camera trajectory 회색 점 : 3D points
	
	

3. Quantitative Results

Metric	COLMAP	MY SfM
# Points (count)	40,708	12,082
Mean (avg error)	0.5697	0.6425
Median	0.4056	0.4384
RMSE	0.7370	0.8651
90th percentile (p90)	1.2706	1.5463
Max error	3.7411	3.2697

4. Analysis

4.1 비교

My SfM shows a slightly larger overall distribution of reprojection errors compared to COLMAP, but the maximum error is lower — indicating that extreme outliers were successfully removed. In

the initial implementation, the mean error exceeded 10, and some points had a maximum error over 3000. In other words, despite using RANSAC, extreme outliers still remained. After inspecting reprojection error metrics (mean/median/max), RMSE, and MAD, and manually removing outliers in post-processing, the maximum error dropped to a level similar to COLMAP, and both the mean error and RMSE were significantly improved.

In addition, unlike COLMAP, My SfM produces fewer triangulated points:

- **My SfM:** performs matching only between adjacent frames and proceeds with incremental SfM
- **COLMAP:** does not restrict itself to adjacent frames and selects the next-best-view by optimizing baseline, viewpoint diversity, and coverage

In other words, while COLMAP considers the entire matching graph, My SfM progresses linearly, which structurally limits opportunities for triangulation. Furthermore, some views failed to be registered during PnP, causing them to be skipped. This likely occurred when the baseline was too small or when the 2D–3D correspondences were of poor quality. Consequently, COLMAP obtains more points through global matching graph exploration and next-best-view selection, whereas My SfM, constrained by local matching and limited baselines, produces fewer triangulated points.

4.2 Challenging

The most significant issue during implementation was the presence of extreme reprojection error outliers. Since RANSAC, PnP, and triangulation were all implemented manually, the robustness was lower compared to OpenCV's internal implementation (which performs additional validation, normalization, and un-normalization steps). Outliers mainly occurred in the following situations:

- When the two projection rays became parallel or degenerate during triangulation
- When triangulation was attempted with an extremely small baseline
- When PnP incorrectly selected the inlier set

As a result, some points exhibited reprojection errors exceeding 3000 pixels, heavily distorting the mean error.

4.3 개선안

Although the issue was resolved by manually removing outliers during post-processing, the following structural improvements can prevent the problem from recurring:

Improvement	Expected Effect
Introduce next-best-view selection (including non-adjacent frames)	Secures larger baselines → increases triangulation stability
PnP with RANSAC + EPnP + refinement (already implemented) + re-select inliers	Prevents incorrect initial pose estimation
Check triangulation feasibility before triangulating (triangulation angle > threshold)	Avoids cases where the baseline is too small
Instead of re-seeding after PnP failure, search neighboring nodes in the matching graph	Reduces skipped views